FUNCTIONAL DEPENDENCES

K Krishna priya

CSE department, IIIT Nuzvid

DATABASE SCHEMA DESIGN

- So far, we have learned database query languages:
- – SQL, Relational Algebra
- • How can you tell whether a given database schema is “good” or “bad”?
- **Design theory:**
- – A set of design principles that allows one to decide what constitutes a “good” or “bad” database schema design.
- – A set of algorithms for modifying a “bad” design to a “better” one.

EXAMPLE

- If we know that rank determines the salary scale, which is a better design? Why?
- Employees(eid, name, addr, rank, salary_scale)

  OR
- Employees(eid, name, addr, rank)
- Salary_Table(rank, salary_scale)

eid	name	addr	rank	salary_scale
34-133	Jane	Elm St.	6	70-90
33-112	Hugh	Pine St.	3	30-40
26-002	Gary	Elm St.	4	35-50
51-994	Ann	South St.	4	35-50
45-990	Jim	Main St.	6	70-90
98-762	Paul	Walnut St.	4	35-50

Lots of duplicate information
– Employees who have the same rank have the same salary scale.

UPDATE ANOMALY

- Update anomaly
- – If one copy of salary scale is changed, then all copies of
- that salary scale (of the same rank) have to be changed.

eid	name	addr	rank	salary_scale
34-133	Jane	Elm St.	6	70-90
33-112	Hugh	Pine St.	3	30-40
26-002	Gary	Elm St.	4	35-50
51-994	Ann	South St.	4	35-50
45-990	Jim	Main St.	6	70-90
98-762	Paul	Walnut St.	4	35-50

INSERTION ANOMALY

- – How can we store a new rank and salary scale information
- if currently, no employee has that rank?
- – Use NULLS?

eid	name	addr	rank	salary_scale
34-133	Jane	Elm St.	6	70-90
33-112	Hugh	Pine St.	3	30-40
26-002	Gary	Elm St.	4	35-50
51-994	Ann	South St.	4	35-50
45-990	Jim	Main St.	6	70-90
98-762	Paul	Walnut St.	4	35-50

DELETION ANOMALY

- If Hugh is deleted, how can we retain the rank and salary
- scale information?
- – Is using NULL a good choice?

eid	name	addr	rank	salary_scale
34-133	Jane	Elm St.	6	70-90
33-112	Hugh	Pine St.	3	30-40
26-002	Gary	Elm St.	4	35-50
51-994	Ann	South St.	4	35-50
45-990	Jim	Main St.	6	70-90
98-762	Paul	Walnut St.	4	35-50

SO WHAT WOULD BE A GOOD SCHEMA DESIGN FOR THIS EXAMPLE?

- salary_scale is dependent only on rank
- – Hence associating employee information such as name,
- addr with salary_scale causes redundancy.
- • Based on the constraints given, we would like to refine the
- schema so that such redundancies cannot occur.
- Note however, that sometimes database designers may choose to live
- with redundancy in order to improve query performance.
- – Ultimately, a good design is depends on the query workload.
- – But understanding anomalies and how to deal with them is still
- important

FUNCTIONAL DEPENDENCIES

- The information that rank determines salary_scale is a type of integrity constraint known as a functional dependency (FD).
- Functional dependencies can help us detect anomalies that may exist in a given schema.
- The FD "rank → salary_scale" suggests that
- Employees(eid, name, addr, rank, salary_scale) should be decomposed into two relations:
- Employees(eid, name, addr, rank)
- Salary_Table(rank, salary_scale).

INTRODUCTION

- In relational database management, functional dependency is a concept that specifies the relationship between two sets of attributes where one attribute determines the value of another attribute.
- It is denoted as **X** → **Y**, where the attribute set on the left side of the arrow, **X** is called **Determinant**, and **Y** is called the **Dependent**.

CONT..

- A Functional Dependency in DBMS is a fundamental concept that describes the relationship between attributes (columns) in a table.
- It shows how the values in one or more attributes determine the value in another.
- it describes how data in one column or set of columns can relate to data in another column. It helps to maintain the quality of the data in DBMS.

WHAT IS FUNCTIONAL DEPENDENCY?

- A functional dependency occurs when one attribute uniquely determines another attribute within a relation.
- It is a constraint that describes how attributes in a table relate to each other. If attribute **A** functionally determines attribute **B** we write this as the **A→B**.
- Functional dependencies are used to mathematically express relations among database entities and are very important to understanding advanced concepts in Relational Database Systems.

MEANING OF AN FD

- Let R be a relation schema. A functional dependency (FD) is an integrity constraint of the form:
- X → Y (read as "X determines Y or X functionally determines Y")
- where X and Y are non-empty subsets of attributes of R.
- A relation instance **r** of **R** satisfies the FD **X → Y** if
- for every pair of tuples t1 and t2 in r, if **t1[X] = t2[X], then t1[Y] = t2[Y].**
- Denotes the X value(s) of tuple t, i.e., project t on the attributes in X.

ILLUSTRATION OF THE SEMANTICS OF AN FD

Relation schema R with the FD A1, …, Am → B1, …, Bn where { A1, …, Am, B1, …, Bn } ⊆ attributes(R).

roll_no	name	dept_name	dept_building
42	abc	CO	A4
43	pqr	IT	A3
44	xyz	CO	A4
45	xyz	IT	A3
46	mno	EC	B2
47	jkl	ME	B2

FROM THE ABOVE TABLE WE CAN CONCLUDE SOME VALID FUNCTIONAL DEPENDENCIES:

- roll_no → { name, dept_name, dept_building },→ Here, roll_no can determine values of fields name, dept_name and dept_building, hence a valid Functional dependency
- roll_no → dept_name , Since, roll_no can determine whole set of {name, dept_name, dept_building}, it can determine its subset dept_name also.
- dept_name → dept_building , Dept_name can identify the dept_building accurately, since departments with different dept_name will also have a different dept_building
- More valid functional dependencies: roll_no → name, {roll_no, name} ⇢ {dept_name, dept_building}, etc.

HERE ARE SOME INVALID FUNCTIONAL DEPENDENCIES:

- name → dept_name Students with the same name can have different dept_name, hence this is not a valid functional dependency.
- dept_building → dept_name There can be multiple departments in the same building. Example, in the above table departments ME and EC are in the same building B2, hence dept_building → dept_name is an invalid functional dependency.
- More invalid functional dependencies: name → roll_no, {name, dept_name} → roll_no, dept_building → roll_no, etc.

ARMSTRONG'S AXIOMS/PROPERTIES OF FUNCTIONAL DEPENDENCIES:

- **Reflexivity:** If Y is a subset of X, then X→Y holds by reflexivity rule
  Example, {roll_no, name} → name is valid.
- **Augmentation:** If X → Y is a valid dependency, then XZ → YZ is also valid by the augmentation rule.
  Example, {roll_no, name} → dept_building is valid, hence {roll_no, name, dept_name} → {dept_building, dept_name} is also valid.
- **Transitivity**: If X → Y and Y → Z are both valid dependencies, then X→Z is also valid by the Transitivity rule.
  Example, roll_no → dept_name & dept_name → dept_building, then roll_no → dept_building is also valid.

UNION AND DECOMPOSITION RULES

- **Union:** If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.
- **Decomposition:** If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.
- Union and Decomposition rules are not essential. In other words, they can be derived using Armstrong's axioms.

Derivation of the Union rule:
Since $X \rightarrow Z$, we get $XY \rightarrow YZ$ (augmentation)
Since $X \rightarrow Y$, we get $X \rightarrow XY$ (augmentation)
Therefore, $X \rightarrow YZ$ (transitivity)

Derivation of the Decomposition rule:
$X \rightarrow YZ$ (given)
$YZ \rightarrow Y$ (reflexivity)
$YZ \rightarrow Z$ (reflexivity)
Therefore, $X \rightarrow Y$ and $X \rightarrow Z$ (transitivity).

PSEUDO-TRANSITIVITY RULE

- **Pseudo-Transitivity:** If X -> Y and WY -> Z, then XW -> Z.
- Can you derive this rule using Armstrong's axioms?

Derivation of the Pseudo-Transitivity rule:
X -> Y and WY -> Z
XW -> WY (augmentation)
WY -> Z (given)
Therefore XW -> Z (transitivity)

TYPES OF FUNCTIONAL DEPENDENCIES IN DBMS

- Trivial functional dependency
- Non-Trivial functional dependency
- Multivalued functional dependency
- Transitive functional dependency

1. TRIVIAL FUNCTIONAL DEPENDENCY

- In **Trivial Functional Dependency**, a dependent is always a subset of the determinant. i.e. If **X → Y** and **Y is the subset of X**, then it is called trivial functional dependency.
- **Example:**

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

- Here, **{roll_no, name} → name** is a trivial functional dependency, since the dependent **name** is a subset of determinant set **{roll_no, name}.** Similarly, **roll_no → roll_no** is also an example of trivial functional dependency.

CONT…

- A trivial functional dependency in DBMS occurs when an attribute or set of attributes (columns) on the left-hand side (LHS) of a functional dependency arrow (->) already determines the attributes on the right-hand side (RHS) without any extra information.
- Suppose we have a table of students with attributes "StudentID" and "StudentName." In this case, if we state the functional dependency as
- StudentID -> StudentName,

- This is a trivial functional dependency. Because within a single "StudentID," there can be only one corresponding "StudentName." In other words, the value of "StudentID" determines the value of "StudentName" without any more information or conditions.

NON-TRIVIAL FUNCTIONAL DEPENDENCY

- In **Non-trivial functional dependency**, the dependent is strictly not a subset of the determinant. i.e. If **X** → **Y** and **Y is not a subset of X**, then it is called Non-trivial functional dependency.

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, roll_no → name is a non-trivial functional dependency, since the dependent name is not a subset of determinant roll_no.
Similarly, {roll_no, name} → age is also a non-trivial functional dependency, since age is not a subset of {roll_no, name}

NON-TRIVAL FD'S

- We want to express a functional dependency based on the student's birthdate (StudentDOB) and class (Class). A non-trivial functional dependency in this case would be

- StudentDOB, Class -> StudentName.

- This functional dependency means that given a combination of a student's date of birth and class, you can uniquely determine their name. It's non-trivial because it provides valuable information about the relationship between attributes in the table.

3. MULTIVALUED FUNCTIONAL DEPENDENCY

- In **Multivalued functional dependency**, entities of the dependent set are **not dependent on each other.** i.e. If **a → {b, c}** and there exists **no functional dependency** between **b and c**, then it is called a **multivalued functional dependency.**

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18
45	abc	19

Here, roll_no → {name, age} is a multivalued functional dependency, since the dependents name & age are not dependent on each other(i.e. name → age or age → name doesn't exist !)

CONT...

- A multivalued functional dependency in a database occurs when one or more attributes determine multiple unrelated sets of values in another attribute.
- It shows that changes in the determining attributes can lead to various combinations of values in the dependent attribute, indicating complex relationships within the data.

Student_ID	Student_Name	Courses_Enrolled
1	Alice	{Math, English}
2	Bob	{Science, History}
3	Carol	{Math, Science}

In this case, the multivalued dependency in DBMS holds:

Alice is Student 1 enrolled in {Math, English}.
Bob is Student 2 enrolled in {Science, History}.
Carol is Student 3 enrolled in {Math, Science}.

TRANSITIVE FUNCTIONAL DEPENDENCY

- In transitive functional dependency, dependent is indirectly dependent on determinant. i.e. If **a** → **b** & **b** → **c**, then according to axiom of transitivity, **a** → **c**. This is a **transitive functional dependency.**

enrol_no	name	dept	building_no
42	abc	CO	4
43	pqr	EC	2
44	xyz	IT	1
45	abc	EC	2

Here, enrol_no → dept and dept → building_no. Hence, according to the axiom of transitivity, enrol_no → building_no is a valid functional dependency.
This is an indirect functional dependency, hence called Transitive functional dependency.

CONT…

- Consider a database table called "Student_Info" with the following attributes:
- Student_ID (unique identifier for each student),
- Student_Name,
- Student_Address
- Student_City.
- In this example, we can assume that Student_Address depends on Student_City, and Student_City depends on Student_ID. This creates a transitive dependency in DBMS, where Student_ID indirectly determines Student_Address.

5. FULLY FUNCTIONAL DEPENDENCY

- In full functional dependency an attribute or a set of attributes uniquely determines another attribute or set of attributes.
- If a relation R has attributes X, Y, Z with the dependencies X->Y and X->Z which states that those dependencies are fully functional.

6. PARTIAL FUNCTIONAL DEPENDENCY

- n partial functional dependency a non key attribute depends on a part of the composite key, rather than the whole key.
- If a relation R has attributes X, Y, Z where X and Y are the composite key and Z is non key attribute. Then X->Z is a partial functional dependency in RBDMS.

ADVANTAGES OF FUNCTIONAL DEPENDENCIES

- Functional dependencies having numerous applications in the field of database management system. Here are some applications listed below:
- **1. Data Normalization**
- **2. Query Optimization**
- **3. Consistency of Data**
- **4. Data Quality Improvement**
- **5. Data Integrity**
- **6. Efficient Storage**
- **7. Ease of Maintenance**

1. DATA NORMALIZATION

- Data normalization is the process of organizing data in a database in order to minimize redundancy and increase data integrity.
- Functional dependencies play an important part in data normalization.
- With the help of functional dependencies we are able to identify the primary key, candidate key in a table which in turns helps in normalization.

QUERY OPTIMIZATION

- With the help of functional dependencies we are able to decide the connectivity between the tables and the necessary attributes need to be projected to retrieve the required data from the tables.
- This helps in query optimization and improves performance.

CONSISTENCY OF DATA

- Functional dependencies ensures the consistency of the data by removing any redundancies or inconsistencies that may exist in the data.
- Functional dependency ensures that the changes made in one attribute does not affect inconsistency in another set of attributes thus it maintains the consistency of the data in database.

DATA QUALITY IMPROVEMENT

- Functional dependencies ensure that the data in the database to be accurate, complete and updated.
- This helps to improve the overall quality of the data, as well as it eliminates errors and inaccuracies that might occur during data analysis and decision making, thus functional dependency helps in improving the quality of data in database.

DATA INTEGRITY

- Functional dependencies help ensure data integrity in a database. By defining rules that govern the relationships between attributes, DBMS can enforce constraints to prevent inconsistent or incorrect data from being entered into the database.

EFFICIENT STORAGE

- Normalizing a database through functional dependencies can lead to more efficient data storage.
- Smaller, normalized tables need less storage space, which can be especially beneficial for large databases.

EASE OF MAINTENANCE

- Databases that adhere to functional dependencies are easier to maintain. When changes are required in the database structure or schema, the impact of those changes is more localized, reducing the risk of introducing errors or inconsistencies in the data.
- Functional dependency in DBMS is the blueprint for organizing and ensuring data accuracy. Identifying these relationships simplifies database design, reduces redundancy, and enhances query efficiency.
- They are the building blocks for maintaining structured and reliable databases, essential for modern data management and information retrieval.
- All the functional dependencies are useful tools to make the Database Management System more efficient for the user.